Definitely Not Wordle

Full-Stack Project Specification — GoLinks 2026 Internship

Frontend	Backend	Hosting	Database
React + TypeScript (Vite)	Python + FastAPI	AWS Lambda + S3 + CloudFront	None (in-memory state)

1. Project Overview

Build a fully functional Wordle clone — *Definitely Not Wordle* — as a demonstration of full-stack engineering skills for the GoLinks 2026 Software Engineering Internship. The project must have a clearly separated frontend and backend, be deployed publicly on AWS, and be version-controlled with a meaningful git history.

Game Rules

- Player must guess a secret **5-letter word** within **5 tries**.
- Any 5-letter string is accepted as a guess (no dictionary enforcement required, but recommended).
- After each guess, every letter receives a **color tile** as feedback:
- **Green** — correct letter, correct position.
- **Yellow** — correct letter, wrong position.
- **Gray** — letter not in the word at all.
- Duplicate-letter edge cases must be handled correctly (extras marked gray, not yellow).
- Player wins by guessing correctly within 5 tries; otherwise the game ends and reveals the answer.

2. Architecture

The app uses a classic client-server split. The React/TypeScript frontend is a static build served from **AWS S3 via CloudFront**. The FastAPI backend runs on **AWS Lambda** (wrapped with Mangum) and is exposed via a **Lambda Function URL** — no API Gateway needed, keeping the setup entirely within the free-forever Lambda tier.

`User Browser`	→	`CloudFront + S3`	`(React/TypeScript static build)`

`User Browser`	→	`Lambda Function URL`	`(FastAPI + Mangum — game API)`
`Lambda`	→	`In-memory dict`	`(session state: secret word + guesses)`

AWS Infrastructure (All Free-Forever Tier)

AWS Service	Role	Free Forever?
S3	Hosts the compiled Vite/React static files	■ Yes
CloudFront	CDN + HTTPS in front of S3 (1 TB/month transfer)	■ Yes
Lambda	Runs FastAPI via Mangum adapter (1M req/month, 400k GB-s)	■ Yes
Lambda Fn URL	Direct HTTPS endpoint — replaces API Gateway (no cost)	■ Yes
API Gateway	NOT used — account is past 12-month free tier	■ Skip
DynamoDB	NOT used — in-memory session state is sufficient for demo	■ Skip

Note: Lambda Function URLs provide a direct HTTPS invoke endpoint at no additional cost. Cold-start latency (~1–2 s) is acceptable for a game demo — open the app a minute before presenting.

3. Repository Structure

```
wordle-clone/
■■■ backend/
■   ■■■ main.py              # FastAPI app + route definitions
■   ■■■ game.py              # Core scoring logic (green/yellow/gray)
■   ■■■ words.py             # Word list (~2 300 answers + ~13 k valid guesses)
■   ■■■ lambda_handler.py    # Mangum wrapper — entry point for AWS Lambda
■   ■■■ requirements.txt     # fastapi, mangum, uvicorn
■
■■■ frontend/
■   ■■■ src/
■   ■   ■■■ components/
■   ■   ■   ■■■ Board.tsx    # 5-row game grid
■   ■   ■   ■■■ Row.tsx      # Single guess row
■   ■   ■   ■■■ Tile.tsx     # Individual letter tile + flip animation
■   ■   ■   ■■■ Keyboard.tsx # On-screen keyboard
■   ■   ■   ■■■ Modal.tsx    # Win / lose overlay
■   ■   ■■■ api.ts           # All fetch() calls to the backend (one file)
■   ■   ■■■ types.ts         # Shared TypeScript interfaces
■   ■   ■■■ App.tsx          # Root component + global state
■   ■   ■■■ main.tsx         # Vite entry point
■   ■■■ index.html
■   ■■■ tsconfig.json
■   ■■■ package.json
■
■■■ .gitignore
■■■ README.md                # Architecture diagram + live demo link
```

4. Backend — FastAPI (Python)

4.1 Scoring Algorithm — game.py

This is the most critical piece of logic. The naive approach (mark yellow if the letter exists anywhere in the secret) produces wrong results when letters repeat. The correct two-pass approach:

- **Pass 1:** Scan all 5 positions. Mark greens first. Track which secret-word positions are consumed.
- **Pass 2:** For remaining letters, check if the letter still has unconsumed occurrences in the secret. If yes → yellow. Otherwise → gray.
- Example: secret=CRANE, guess=CREED → C=green, R=green, E=yellow, E=gray, D=gray (only one E in secret, already used by position 2).

4.2 API Endpoints

Method	Endpoint	Request Body	Response
POST	/api/new-game	{}	{ "session_id": "uuid4" }

Method	Endpoint	Request Body	Response
POST	/api/guess	{ "session_id": "...", "guess": "crane" }	{ "result": [{letter, color}], "status": "playing|won|lost", "guesses_left": 4 }
GET	/api/answer	?session_id=...	{ "answer": "crane" } — required by spec
GET	/api/health	—	{ "ok": true }

*The **/api/answer** endpoint is explicitly required by the GoLinks project spec. It should always return the secret word for the given session, regardless of game status.*

4.3 Session State (In-Memory)

A module-level Python dict stores active sessions keyed by UUID. No database required.

```
sessions: dict[str, dict] = {}
# Each session: { 'secret': str, 'guesses': list[str], 'status': 'playing'|'won'|'lost' }
```

Sessions reset on Lambda cold start. This is fine for a demo — a new game is always one click away.

4.4 Lambda Deployment (lambda_handler.py)

```
from mangum import Mangum
from main import app

handler = Mangum(app, lifespan='off')
```

Deploy as a ZIP package (or container image). Set handler to **lambda_handler.handler** in the Lambda console. Enable a Function URL with CORS allowed for your CloudFront domain.

5. Frontend — React + TypeScript (Vite)

5.1 Component Tree

App.tsx	Root. Owns all global state. Orchestrates API calls.
Board.tsx	Renders 5 Row components. Passes current guess + submitted guesses.
Row.tsx	Renders 5 Tile components for one guess attempt.
Tile.tsx	Single letter square. Applies color class + CSS flip animation on reveal.
Keyboard.tsx	On-screen QWERTY keyboard. Keys reflect best color seen so far.
Modal.tsx	Win/lose overlay. Shows answer on loss, share button on win.
api.ts	All fetch() calls in one file. Exports: newGame(), submitGuess(), getAnswer().
types.ts	Shared interfaces: TileResult, GameStatus, GuessResponse, etc.

5.2 Global State (App.tsx)

```
// Stored in React useState
sessionId:    string | null       // persisted in localStorage
guesses:      TileResult[][]      // submitted rows — drives Board + Keyboard colors
currentGuess: string              // letters typed so far (max 5)
gameStatus:   'playing'|'won'|'lost'
secretWord:   string | null       // fetched from /api/answer after game ends
```

5.3 Key UX Details

- Physical keyboard input (keydown listener) AND on-screen keyboard both work.
- Tiles flip with a CSS 3-D animation on guess submission — staggered by column index.
- Keyboard keys update to green/yellow/gray based on the *best* result seen for each letter across all guesses.
- Invalid-length guess shakes the current row (CSS animation) instead of submitting.
- On game end: Modal shows win/loss message + secret word. New Game button calls /api/new-game.
- sessionId is written to localStorage on new game and read on page load — resuming a session in progress.

5.4 Environment Configuration

```
# .env.production
VITE_API_URL=https://.lambda-url.us-east-1.on.aws

# api.ts reads:
const BASE = import.meta.env.VITE_API_URL
```

6. Git Commit Plan

GoLinks explicitly checks commit history. Commit after each meaningful milestone — never in one big dump at the end.

#	Commit Message	What's Included
1	chore: init repo, add README + .gitignore	git init, folder structure, .gitignore (node_modules, __pycache__, .env, dist)
2	feat(backend): word list + scoring logic	words.py with answer list + valid-guess list. game.py with two-pass scoring algorithm. Unit tests for edge cases (duplicate letters, all-gray, all-green).
3	feat(backend): FastAPI routes	main.py with all 4 endpoints. Testable via Swagger UI at /docs. CORS configured.
4	feat(backend): Lambda handler	lambda_handler.py (Mangum wrapper). requirements.txt. Confirmed working locally with uvicorn.
5	feat(frontend): scaffold + Board layout	Vite + React + TypeScript init. Board, Row, Tile components — static layout, no logic. Basic CSS (dark Wordle theme).
6	feat(frontend): Keyboard component	QWERTY on-screen keyboard. Physical keydown listener. currentGuess state wired up.
7	feat(frontend): connect to backend	api.ts with newGame() + submitGuess(). Full game loop playable end-to-end. sessionId persisted in localStorage.
8	feat(frontend): win/lose modal + answer reveal	Modal component. getAnswer() call on game end. New Game button.
9	feat(frontend): keyboard colors + tile animations	Keyboard keys update to green/yellow/gray. CSS 3-D flip animation on tile reveal.
10	chore: deploy to AWS	Lambda ZIP uploaded. Function URL enabled + CORS set. Frontend built (npm run build) and synced to S3. CloudFront distribution created.
11	docs: update README with demo link + architecture	Live URL, architecture diagram, local dev setup instructions, environment variable docs.

7. Local Development Setup

Backend

```
cd backend
uv venv && source .venv/bin/activate
uv pip install -r requirements.txt
uvicorn main:app --reload --port 8000
# Swagger UI → http://localhost:8000/docs
```

Frontend

```
cd frontend
npm install
echo 'VITE_API_URL=http://localhost:8000' > .env.local
npm run dev
# App → http://localhost:5173
```

8. AWS Deployment Checklist

Backend (Lambda)

- Create Lambda function (Python 3.12 runtime, x86_64).
- Set handler to **lambda_handler.handler**.
- Upload ZIP: backend/ directory + site-packages from venv.
- Enable **Function URL** (auth: NONE). Note the generated URL.
- Add CORS allowed origins: your CloudFront domain (or * during development).
- Test via the Lambda console Test tab and via curl.

Frontend (S3 + CloudFront)

- Set **VITE_API_URL** to your Lambda Function URL, then run **npm run build**.
- Create S3 bucket — disable 'Block all public access'. Enable static website hosting.
- Upload **dist/** contents to the bucket root.
- Create CloudFront distribution pointing to the S3 bucket. Set default root object to **index.html**.
- Add a CloudFront error page: 403/404 → /index.html (required for React Router if used).
- Wait ~5 min for distribution to deploy. Test the CloudFront URL.

9. Requirement Checklist

■	All game rules implemented (green/yellow/gray, 5 tries, win/lose)
■	GET /api/answer endpoint present and returns secret word
■	Any 5-letter string accepted as guess
■	git init before first commit
■	Committed to public GitHub repo
■	Separate frontend (React/TS) and backend (FastAPI/Python)
■	Hosted publicly on AWS (S3 + CloudFront + Lambda)

■	Meaningful git history — 11 commits across development
■	Duplicate-letter edge case handled correctly
■	sessionId persisted in localStorage (resume on refresh)

Good luck, Rahul. Ship early, commit often, and have fun with it.